Guia de Estudo — Previsão de Churn no Ginásio

Oct 15, 2026 · @Alisson Neres Alencar

1. Como usar este guia

Este guia explica, do zero, tudo o que foi feito no projeto de previsão de churn do ginásio. Cada conceito vem com uma explicação simples, um exemplo do dia a dia e o número real do nosso projeto. No fim há perguntas que o professor pode fazer, com respostas prontas.

O projeto numa história. Imagine que você é dono de um ginásio com 4000 alunos. Todo mês, alguns cancelam a matrícula e você só descobre quando já foram embora. Você tem uma planilha com informações de cada aluno: idade, tipo de contrato, quantas vezes vai às aulas, há quanto tempo é cliente.

A pergunta é: **dá para olhar para essa planilha e adivinhar, antes de acontecer, quem vai cancelar no próximo mês?** Se der, você liga para esses alunos, oferece uma aula grátis ou um desconto, e tenta mantê-los.

Foi exatamente isso que construímos: um programa em Python que aprende com o histórico de 4000 alunos e, para cada aluno, diz a probabilidade de ele sair. Ele acerta 82,5% dos alunos que realmente saem.

Ordem sugerida de estudo:

1. Secções 2 a 4: o problema, o que é Machine Learning e os dados.
2. Secções 5 e 6: como o projeto foi organizado (CRISP-DM e planeamento).
3. Secções 7 a 11: o coração técnico (pipeline, fuga de informação, pré-processamento, modelo, baseline).
4. Secções 12 e 13: como medimos se o modelo é bom e o que ele nos ensinou.
5. Secções 14 a 16: reprodutibilidade, perguntas do professor e glossário.

2. O que é churn

Churn é o cliente que cancela. A palavra vem do inglês e significa “agitar” ou “rotatividade”. Em negócios por assinatura, a *taxa de churn* é a percentagem de clientes que saem num período.

Exemplos do dia a dia:

- **Netflix ou Spotify:** você cancela a assinatura porque já não usa. Para a empresa, você virou churn.

- **Operadora de telemóvel:** você troca de operadora por uma oferta melhor. É churn.
- **Ginásio:** você paga três meses, vai cada vez menos e, no quarto mês, não renova. É o caso do nosso projeto.

Porque importa. Conquistar um cliente novo custa publicidade, promoções e tempo de vendas. Manter um cliente atual costuma ser muito mais barato. Por isso, saber *antes* quem está prestes a sair vale dinheiro.

No nosso projeto: dos 4000 clientes, 1061 saíram no mês seguinte. Isso dá uma taxa de churn de **26,5%**, ou seja, mais de 1 em cada 4 alunos.

Pense assim: o nosso modelo é como um "alarme de incêndio" para clientes. Ele não apaga o fogo (não retém ninguém sozinho), mas avisa a equipa do ginásio onde está o fumo, para ela agir a tempo.

3. IA, Machine Learning e classificação

Inteligência Artificial (IA) é o nome geral para programas que fazem tarefas que parecem exigir inteligência: reconhecer rostos, traduzir, recomendar filmes, prever coisas.

Machine Learning (aprendizagem automática) é uma parte da IA em que o programa *aprende com exemplos*, em vez de alguém escrever as regras à mão.

Analogia da criança e das frutas. Ninguém ensina a uma criança a regra "maçã = redonda, vermelha, 7 cm". Mostramos muitas maçãs e muitas laranjas, dizendo o nome de cada uma. Com o tempo, ela reconhece sozinha uma maçã que nunca viu. O Machine Learning faz o mesmo: mostramos milhares de exemplos com a resposta certa e ele descobre o padrão.

Os três ingredientes:

Ingrediente	O que é	No nosso projeto
Exemplos	Os casos de onde se aprende	Os 4000 clientes
Variáveis (features)	As características de cada exemplo	Idade, contrato, frequência, antiguidade… (13)
Variável-alvo (target)	A resposta que queremos prever	Churn: saiu (1) ou ficou (0)

Aprendizagem supervisionada. Chama-se assim porque, durante o treino, o modelo tem um "professor": para cada exemplo, sabemos a resposta certa (se o cliente saiu ou não). O modelo erra, compara com a resposta e ajusta-se.

Classificação vs regressão:

- **Classificação:** a resposta é uma *categoria*. Ex.: "o e-mail é spam ou não?", "o cliente sai ou fica?". É o nosso caso. Como só há duas categorias, chama-se **classificação binária**.
- **Regressão:** a resposta é um *número contínuo*. Ex.: "quanto vai custar esta casa?", "quantos alunos vêm amanhã?".

Modelo é o resultado do treino: uma "fórmula" aprendida que recebe as características de um cliente novo e devolve uma previsão. **Treinar** é o processo de ajustar essa fórmula aos exemplos.

4. Os dados: cada coluna explicada

O dataset é o *Gym customers features and churn*, do Kaggle (um site público de datasets). É um ficheiro CSV — uma tabela em texto, como uma folha de Excel — com **4000 linhas (clientes) e 14 colunas**. Não tem valores em falta, linhas repetidas nem números de identificação.

Coluna	O que significa	Tipo	Ligação ao churn
gender	Género (0/1)	Binária	Nenhuma (r = 0,00)
Near_Location	Mora ou trabalha perto do ginásio	Binária	Fraca (−0,13)
Partner	Trabalha numa empresa parceira (desconto corporativo)	Binária	Fraca (−0,16)
Promo_friends	Entrou pela promoção "traz um amigo"	Binária	Fraca (−0,16)
Phone	Deu número de telefone	Binária	Nenhuma (0,00)
Group_visits	Frequenta aulas de grupo	Binária	Fraca (−0,18)
Contract_period	Duração do contrato: 1, 6 ou 12 meses	Categórica	Média (−0,39)
Age	Idade	Contínua	Média (−0,41)

Coluna	O que significa	Tipo	Ligação ao churn
Avg_additional_charges_total	Gasto médio extra (bar, massagens, loja)	Contínua	Fraca (−0,20)
Month_to_end_contract	Meses que faltam para o contrato acabar	Contínua	Média (−0,38)
Lifetime	Antiguidade: meses desde que se inscreveu	Contínua	A mais forte (−0,44)
Avg_class_frequency_total	Média de idas por semana, desde sempre	Contínua	Fraca (−0,25)
Avg_class_frequency_current_month	Média de idas por semana, no mês atual	Contínua	Forte (−0,41)
Churn	**Saiu no mês seguinte (1) ou ficou (0)**	**Alvo**	—

Três tipos de variáveis, três tratamentos:

- **Binária:** só tem dois valores, 0 ou 1 (sim/não). Ex.: "frequenta aulas de grupo?".
- **Categórica:** escolhe entre categorias. A duração do contrato é escrita com números (1, 6, 12), mas é um *tipo de plano*, como "mensal, semestral, anual". Por isso tratámo-la como categoria.
- **Contínua:** um número que pode variar muito. Ex.: idade 19, 27, 34…

O que é a correlação (r). É um número entre −1 e +1 que mede se duas coisas andam juntas. Perto de **+1**: quando uma sobe, a outra sobe (altura e peso). Perto de **−1**: quando uma sobe, a outra desce (quilometragem do carro e preço de revenda). Perto de **0**: não há relação (número do sapato e nota no exame).

Leitura da tabela: todas as ligações são negativas. Quanto mais antigo, mais frequente, mais velho e com contrato mais longo, *menos* o cliente sai. Género e telefone não dizem nada sobre o churn.

Desequilíbrio de classes: 73,5% ficaram e 26,5% saíram. As duas "classes" não têm o mesmo tamanho. Isto vai ser muito importante nas métricas (secção 12).

5. CRISP-DM: o roteiro do projeto

CRISP-DM (*Cross-Industry Standard Process for Data Mining*) é um roteiro de 6 fases, criado em 2000, que quase todas as equipas de dados usam para organizar um projeto. Não é um programa: é uma *ordem de trabalho*.

Analogia: abrir um restaurante. Primeiro percebe-se o que o bairro quer comer, depois vê-se que ingredientes há, prepara-se, cozinha-se, prova-se e só então se serve. Se o prato não agrada, volta-se atrás. O CRISP-DM é isso para dados.

Fase	Pergunta que responde	Analogia do restaurante	O que fizemos	Decisão tomada
1. Compreensão do negócio	Que problema queremos resolver?	O que o bairro quer comer?	Definimos prever quem cancela no mês seguinte	Classificação binária; dar prioridade ao recall
2. Compreensão dos dados	Que dados temos e o que dizem?	Que ingredientes há na despensa?	Analisámos as 14 colunas, omissos, correlações	Manter todas as variáveis; contrato como categoria
3. Preparação dos dados	Como deixar os dados prontos?	Lavar, cortar, temperar	Limpeza, divisão 80/20, imputação, one-hot, normalização	Incluir imputação mesmo sem omissos
4. Modelação	Que modelo aprende o padrão?	Cozinhar o prato	Regressão Logística num Pipeline + baseline	Hiperparâmetros por omissão, limiar 0,5
5. Avaliação	O modelo é bom o suficiente?	Provar antes de servir	Matriz de confusão, precision, recall, F1	Critério cumprido (recall 0,825)
6. Implementação	Como é usado na prática?	Servir ao cliente	Guardar com joblib; lista mensal de risco	Monitorizar e retreinar se o recall cair

Detalhe importante: o CRISP-DM é um *ciclo*, não uma linha reta. Se na avaliação o modelo fosse fraco, voltávamos à preparação ou até à compreensão do negócio. O enunciado pediu, para cada fase, um parágrafo com *o que foi feito* e *a decisão tomada* — é exatamente a última coluna da tabela.

6. Planeamento: ficha de projeto, mini-sprints e riscos

Ficha de projeto é o "BI" do projeto: em poucas linhas diz o que vamos fazer, para quem e como saberemos se deu certo. Serve para todos na equipa terem a mesma ideia do objetivo.

Campo	Pergunta	A nossa resposta
Problema	O que queremos resolver?	Prever quem cancela no mês seguinte
Utilizador	Quem vai usar o resultado?	A equipa de retenção do ginásio
Decisão apoiada	Que decisão o modelo ajuda a tomar?	Quem contactar primeiro este mês
Variável-alvo	O que o modelo prevê?	Churn (1 = sai, 0 = fica)
Critério de sucesso	Quando dizemos "deu certo"?	Recall ⩾ 0,75 e F1 muito acima do baseline
Fora do âmbito	O que não vamos fazer?	Campanhas de retenção, produção, outros algoritmos

Porque definir o critério de sucesso antes? É como combinar a nota mínima antes do exame. Se definíssemos depois de ver os resultados, seria fácil "batotar" e dizer que qualquer resultado é bom.

Porque dizer o que fica fora do âmbito? Para o projeto não crescer sem fim. Numa obra, se não se diz "só vamos pintar a sala", de repente está a remodelar a casa toda.

Mini-sprints. "Sprint" vem das metodologias ágeis: um bloco curto de trabalho com um entregável no fim. Em vez de fazer tudo de uma vez e só ver o resultado no último dia, entrega-se algo concreto a cada etapa.

1. **Planeamento e desenho** → entregável: ficha de projeto, diagrama do pipeline, estrutura do relatório.
2. **Núcleo funcional** → entregável: código que treina o modelo de forma reprodutível.
3. **Integração, validação e apresentação** → entregável: métricas, modelo guardado, relatório, README e apresentação.

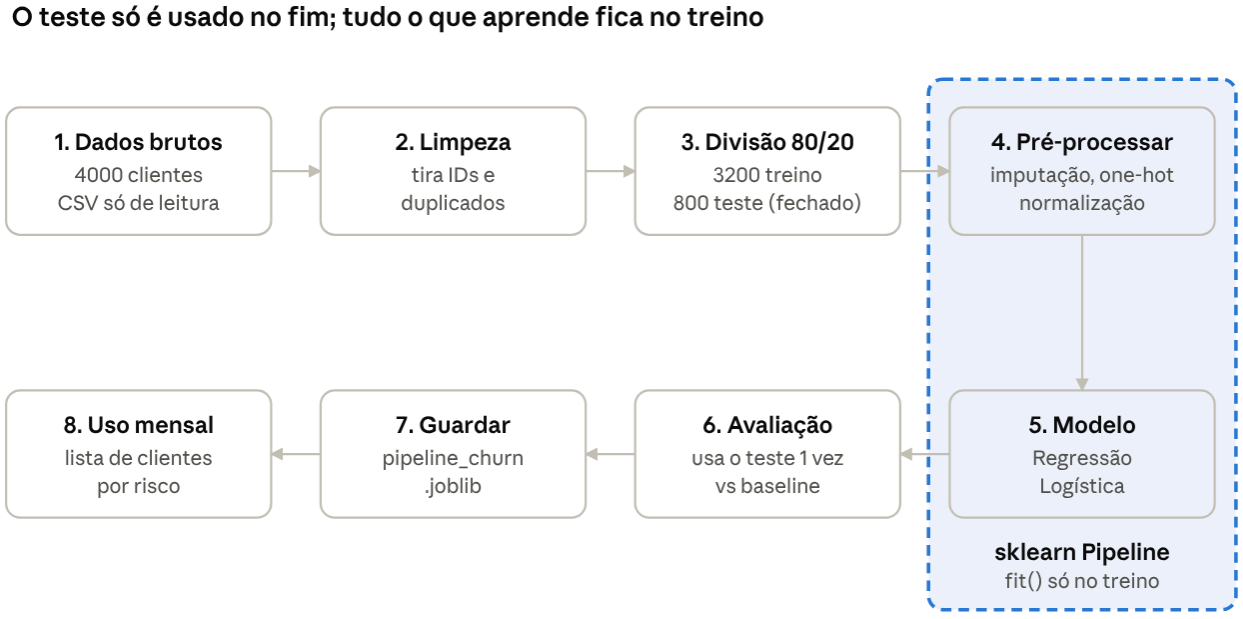
Riscos e mitigação. Um *risco* é algo que pode correr mal; a *mitigação* é o que fazemos para evitar ou reduzir o dano. É como levar guarda-chuva quando a previsão diz 60% de chuva.

Risco	Exemplo do que podia correr mal	Mitigação
Fuga de informação	O modelo “espreita” o teste e parece melhor do que é	Dividir antes de transformar; tudo dentro do Pipeline
Classes desequilibradas	Um modelo que diz sempre “fica” acerta 73,5% e não serve para nada	Stratify; medir recall e F1, não accuracy
Variáveis correlacionadas	Coeficientes com sinais estranhos e conclusões erradas	Normalizar; ler coeficientes em conjunto; validação cruzada

Papéis: Alisson ficou com dados, código e modelo; Giuliana com relatório, ficheiros e apresentação, com revisão cruzada.

7. O pipeline passo a passo

Pipeline (“tubagem” ou “linha de montagem”) é a sequência fixa de passos que transforma dados brutos num modelo pronto. Como numa fábrica de carros: a peça entra de um lado, passa por estações sempre na mesma ordem e sai pronta do outro.



pipeline do projeto · 8 passos, o Pipeline do scikit-learn a tracejado

A caixa a tracejado (passos 4 e 5) é o objeto `Pipeline` do scikit-learn: pré-processamento e modelo colados num só bloco, ajustado só com os dados de treino.

Cada passo, em linguagem simples:

1. **Dados brutos:** ler o CSV tal como veio. Nunca o alteramos — como uma fotografia original que se guarda e só se trabalha em cópias.
2. **Limpeza:** tirar o que não serve (IDs, linhas repetidas). Aqui não se calcula nada a partir dos dados.
3. **Divisão treino/teste:** separar 80% para aprender e 20% para testar no fim (secção 8).
4. **Pré-processamento:** pôr os dados num formato que o modelo entende (secção 9).
5. **Modelo:** a Regressão Logística aprende o padrão (secção 10).
6. **Avaliação:** medir o desempenho nos 800 clientes de teste e comparar com o baseline (secções 11 e 12).
7. **Guardar:** gravar o pipeline inteiro num ficheiro `.joblib` (secção 14).
8. **Uso mensal:** no início de cada mês, passar os clientes atuais pelo modelo e entregar uma lista ordenada por risco.

Porque juntar pré-processamento e modelo num só Pipeline? Porque assim é impossível esquecer um passo ou fazê-lo na ordem errada quando chegar um cliente novo. É como uma máquina de café de cápsulas: você só põe a cápsula, a máquina faz moer, dosear e aquecer sempre igual.

8. Divisão treino/teste e fuga de informação

Porque dividir os dados? Para saber se o modelo aprendeu de verdade ou só decorou. Testar um modelo com os mesmos dados com que aprendeu é como dar a um aluno o exame com as mesmas perguntas da ficha de exercícios: ele tira 20, mas não prova nada.

- **Treino (80% = 3200 clientes):** os exercícios resolvidos com que o modelo estuda.
- **Teste (20% = 800 clientes):** o exame final, com perguntas que ele nunca viu.

Os três parâmetros da divisão:

- `test_size=0.20` — 20% vão para teste.
- `random_state=42` — a "semente" do sorteio. A divisão é aleatória, mas com a mesma semente sai *sempre o mesmo sorteio*. É como baralhar cartas sempre da mesma maneira: qualquer pessoa que corra o código obtém exatamente os mesmos resultados. O 42 é só uma convenção (vem do livro *À Boleia pela Galáxia*); qualquer número fixo serve.
- `stratify=y` — *estratificação*: garante que o treino e o teste têm a mesma proporção de churn (26,5% em ambos). Sem isto, por azar, o teste podia ficar com 15% ou 35% de churn e as métricas ficavam distorcidas. É como uma sondagem eleitoral que garante que a amostra tem a mesma proporção de homens e mulheres do país.

Fuga de informação (*data leakage*) acontece quando, durante o treino, o modelo tem acesso a informação que não teria na vida real — normalmente, informação do conjunto de teste. O resultado parece ótimo no papel e falha no mundo real.

Exemplo concreto de fuga (o erro que evitámos). Para normalizar a idade, é preciso calcular a média das idades. Se calcularmos essa média com *todos* os 4000 clientes antes de dividir, a média já "viu" os 800 clientes de teste. É pouco, mas é o aluno a espreitar o enunciado do exame. O correto é calcular a média **só com os 3200 de treino** e aplicar essa mesma média ao teste.

Outro exemplo, mais grave: usar como variável "data do cancelamento" para prever se o cliente cancela. O modelo acertaria 100%, mas na vida real essa informação só existe *depois* de o cliente sair.

Os 4 pontos onde evitámos a fuga:

1. A limpeza não calcula estatísticas (só tira IDs e duplicados).
2. A divisão é feita **antes** de qualquer transformação que aprenda com os dados.
3. Imputação, codificação e normalização estão dentro do `Pipeline`: o `.fit()` só vê o treino; no teste só se faz `.transform()`.
4. O teste é usado **uma única vez**, no fim. Não fomos ajustando o modelo a olhar para ele.

fit vs transform (decore isto): `fit` = *aprender* (calcular médias, medianas, coeficientes). `transform` = *aplicar* o que foi aprendido. Regra de ouro: **fit só no treino, transform no treino e no teste.**

Ponto de discussão: as colunas do "mês atual" (`Avg_class_frequency_current_month`, `Month_to_end_contract`) só são legítimas se forem conhecidas *no momento da previsão*. Assumimos que o modelo corre no início do mês para prever o mês seguinte. Se o professor perguntar, esta é a resposta.

9. Pré-processamento: imputação, one-hot e normalização

Modelos matemáticos só entendem números, completos e numa escala comparável. O pré-processamento é a "mise en place" da cozinha: deixar tudo lavado, cortado e medido antes de cozinhar.

Imputação (preencher valores em falta)

Se um cliente não tem a idade preenchida, o modelo não consegue calcular nada. *Imputar* é preencher esse buraco com um valor razoável.

- **Mediana** (para números contínuos): o valor do meio quando se ordenam todos. Se as idades forem 20, 22, 25, 30 e 70, a mediana é 25. Usámos a mediana e não a média porque a média (33,4) é "puxada" pelo valor extremo 70. Pense no salário médio de uma sala onde entra um milionário: a média dispara, a mediana quase não muda.
- **Moda** (para categorias e binárias): o valor mais frequente. Se a maioria tem contrato de 1 mês, um contrato em falta vira "1 mês".

No nosso dataset não havia valores em falta. Mesmo assim, incluímos a imputação para que o modelo guardado não "rebente" se um dia chegar um cliente novo com um campo vazio. É um cinto de segurança.

Codificação one-hot (transformar categorias em colunas 0/1)

A duração do contrato é 1, 6 ou 12. Se deixássemos como número, o modelo pensaria que "12 é o dobro de 6" e que existe uma ordem matemática entre planos. Mas são *tipos de plano*, como "mensal, semestral, anual".

O **one-hot** cria uma coluna por categoria, com 1 na categoria do cliente e 0 nas outras:

Contrato original	Contract_period_1	Contract_period_6	Contract_period_12
1 mês	1	0	0
6 meses	0	1	0
12 meses	0	0	1

É como um formulário com caixinhas para assinalar: só uma fica com o visto. `handle_unknown="ignore"` diz: se aparecer um contrato que nunca vimos (ex.: 3 meses), põe tudo a 0 em vez de dar erro.

Normalização (StandardScaler)

A idade vai de 18 a 41; os gastos extra vão de 0 a 552. Sem normalizar, o modelo trataria "552" como muito mais importante do que "41" só porque o número é maior. É como comparar uma distância em metros com outra em quilómetros sem converter.

O **StandardScaler** transforma cada valor assim:

```
z = \frac{x - \text{média}}{\text{desvio-padrão}}
```

Exemplo: se a idade média é 29 e o desvio-padrão é 3, um cliente de 32 anos fica com z = (32 − 29) / 3 = **+1**: "um desvio-padrão acima da média". Um de 26 fica com **−1**. Depois disto, todas as variáveis falam a mesma "língua": 0 = normal, +1 = acima, −1 = abaixo.

Desvio-padrão mede o quão espalhados estão os valores. Turma com notas 9, 10, 11: desvio pequeno. Turma com notas 2, 10, 18: desvio grande, mesma média.

Bónus importante: como tudo fica na mesma escala, podemos comparar os coeficientes do modelo entre si (secção 13).

ColumnTransformer: o "distribuidor"

O `ColumnTransformer` encaminha cada grupo de colunas para o tratamento certo, como uma triagem de correio:

Grupo	Colunas	Imputação	Transformação
Contínuas	Age, Lifetime, frequências, gastos, meses para o fim (6)	Mediana	StandardScaler
Categórica	Contract_period (1)	Moda	OneHotEncoder
Binárias	gender, Near_Location, Partner, Promo_friends, Phone, Group_visits (6)	Moda	Nenhuma (já são 0/1)

10. Regressão Logística

Apesar do nome, a Regressão Logística é um algoritmo de *classificação*. Ela devolve uma probabilidade entre 0 e 1 (ex.: "78% de chance de sair") e, a partir dela, decide a classe.

Como funciona, em 3 passos:

1. **Soma pesada (pontuação).** Cada variável recebe um *peso* (coeficiente). O modelo multiplica cada valor do cliente pelo seu peso e soma tudo. Pense num sistema de pontos de risco: "foi poucas vezes este mês: +3 pontos"; "é cliente há 2 anos: −4 pontos".
2. **Transformar a pontuação em probabilidade.** A pontuação pode ir de −∞ a +∞, mas uma probabilidade tem de ficar entre 0 e 1. Para isso usa-se a função **sigmoide**, que tem forma de "S": pontuações muito negativas dão quase 0, muito positivas dão quase 1, e 0 dá exatamente 0,5.
3. **Decidir com um limiar.** Se a probabilidade ≥ 0,5, o modelo diz "vai sair" (1); senão, "fica" (0). Usámos o limiar padrão de 0,5.

```
P(\text{churn}) = \frac{1}{1 + e^{-(b_0 + b_1 x_1 + b_2 x_2 + \dots + b_n x_n)}}
```

Onde *x* são as características do cliente (já normalizadas), *b* são os coeficientes aprendidos e *b0* é o ponto de partida (intercepto).

Analogia do termómetro com alarme. A soma pesada é a temperatura; a sigmoide converte-a numa escala de 0 a 100% de "perigo"; o limiar é o ponto a partir do qual o alarme toca.

O que é "treinar" aqui? É encontrar os pesos *b* que fazem as probabilidades previstas ficarem o mais perto possível da realidade nos 3200 clientes de treino. O computador ajusta os pesos aos poucos até o erro parar de diminuir. `max_iter=1000` é o número máximo de tentativas de ajuste.

Porque escolhemos este algoritmo:

- **Simples e rápido:** treina em segundos.
- **Interpretável:** cada coeficiente diz *quanto e em que direção* uma variável mexe no risco. Um gestor de ginásio consegue perceber.
- **Foi o pedido pelo enunciado** para problemas de classificação.

Limitações: assume que o efeito de cada variável é "linear" (sempre na mesma direção e na mesma força). Relações complicadas, do tipo "só os jovens com contrato curto saem", seriam melhor apanhadas por algoritmos como árvores de decisão — que ficaram fora do âmbito.

Regressão Logística vs Regressão Linear: a linear prevê um número qualquer (ex.: preço de uma casa); a logística prevê uma probabilidade entre 0 e 1 para decidir entre duas classes. Se o problema fosse de regressão, o enunciado pedia Regressão Linear.

11. Baseline e validação cruzada

Baseline (DummyClassifier)

Baseline é a "fasquia mínima": um modelo propositadamente burro para compararmos. Se o nosso modelo não for claramente melhor do que ele, não vale a pena.

O `DummyClassifier(strategy="most_frequent")` não olha para nenhuma variável: responde *sempre* a classe mais comum. Como 73,5% dos clientes ficam, ele diz sempre "fica".

Analogia: um meteorologista preguiçoso em Lisboa que diz todos os dias "não vai chover". Acerta na maioria dos dias — mas nunca avisa da chuva, que é justamente o que interessa.

Resultado do baseline no nosso teste: 73,5% de *accuracy* (parece bom!), mas **0 clientes de churn detetados** (recall = 0, F1 = 0). É a melhor prova de que a accuracy engana quando as classes são desequilibradas.

Validação cruzada (5 folds)

Problema: uma única divisão treino/teste pode ter sorte ou azar. E se, por acaso, os 800 clientes de teste forem os mais fáceis?

Solução: a validação cruzada *k-fold*. Só dentro dos 3200 de treino:

1. Divide-se o treino em 5 partes iguais ("folds"), cada uma com 640 clientes.
2. Treina-se com 4 partes e testa-se na 5.ª.
3. Repete-se 5 vezes, cada vez com uma parte diferente como teste.
4. Calcula-se a média e a variação dos 5 resultados.

Analogia: em vez de julgar um jogador de futebol por um único jogo, vê-se 5 jogos diferentes. Se ele joga bem em todos, é consistente; se num marca 3 golos e noutro joga mal, é instável.

O nosso resultado: F1 por fold = 0,873; 0,857; 0,852; 0,878; 0,831. Média **0,858 ± 0,017**. O "± 0,017" (desvio-padrão) é pequeno, ou seja, o modelo é **estável**: não depende da sorte de uma divisão.

Porquê só no treino? Para o teste continuar "virgem" até ao fim (regra 4 da fuga de informação). E usou-se `StratifiedKFold`: cada fold mantém os 26,5% de churn.

12. As métricas, uma a uma

Tudo começa na matriz de confusão. É uma tabela que cruza o que *realmente aconteceu* com o que o *modelo previu*. Estes são os números reais do nosso modelo nos 800 clientes de teste:

Real ↓ / Previsto →	Previu "fica"	Previu "sai"
Ficou de facto (588)	**565** — Verdadeiro Negativo (VN)	**23** — Falso Positivo (FP)
Saiu de facto (212)	**37** — Falso Negativo (FN)	**175** — Verdadeiro Positivo (VP)

Como ler os quatro quadrados (analogia do teste de gravidez):

- **VP (175):** o teste diz "grávida" e está. O modelo disse "vai sair" e saiu. ✔
- **VN (565):** o teste diz "não grávida" e não está. Disse "fica" e ficou. ✔
- **FP (23) — falso alarme:** o teste diz "grávida" e não está. Disse "vai sair", mas ficou. Custo: uma chamada ou um desconto desnecessário.
- **FN (37) — o que escapou:** o teste diz "não grávida" e está. Disse "fica", mas saiu. Custo: perdemos o cliente sem tentar retê-lo. **É o erro mais caro para o ginásio.**

Truque de memória: *Positivo/Negativo* = o que o modelo disse; *Verdadeiro/Falso* = se acertou.

Accuracy (exatidão)

Pergunta: de todas as previsões, quantas acertou?

```
\text{Accuracy} = \frac{VP + VN}{\text{total}} = \frac{175 + 565}{800} = 0{,}925
```

Porque não a usámos como métrica principal: o baseline, que nunca deteta ninguém, já tem 0,735. Quando 3 em cada 4 clientes ficam, "acertar muito" é fácil sem servir para nada. É o meteorologista que diz sempre "não chove".

Precision (precisão)

Pergunta: dos clientes que o modelo apontou como "vai sair", quantos saíram mesmo?

```
\text{Precision} = \frac{VP}{VP + FP} = \frac{175}{175 + 23} = 0{,}884
```

Leitura: quando o modelo dá o alarme, acerta 88,4% das vezes. **Analogia:** um detetor de fumo que só apita quando há mesmo fumo (e não quando fazemos torradas) tem precision alta. Precision baixa = a equipa perde tempo a ligar para quem nunca ia sair.

Recall (sensibilidade)

Pergunta: dos clientes que realmente saíram, quantos o modelo apanhou?

```
\text{Recall} = \frac{VP}{VP + FN} = \frac{175}{175 + 37} = 0{,}825
```

Leitura: o modelo apanhou 82,5% dos que saíram. **Analogia:** uma rede de pesca. Recall alto = a rede apanha quase todos os peixes que queremos, mesmo que venha algum lixo junto. Recall baixo = os peixes fogem pelos buracos.

Porque é a métrica mais importante no nosso projeto: deixar escapar um cliente que vai sair (FN) custa uma mensalidade perdida para sempre; um falso alarme (FP) custa só um telefonema. Por isso o critério de sucesso foi **recall ≥ 0,75**.

O equilíbrio entre precision e recall

As duas puxam em sentidos opostos. Se o modelo disser que *todos* vão sair, o recall é 100% mas a precision cai para 26,5%. Se só apontar o cliente mais óbvio, a precision é 100% mas o recall é quase 0. Mexer no **limiar** (0,5) desloca este equilíbrio: baixá-lo para 0,3 aumenta o recall e baixa a precision.

F1-score

Pergunta: qual é o equilíbrio entre precision e recall, num só número?

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = 2 \times \frac{0{,}884 \times 0{,}825}{0{,}884 + 0{,}825} = 0{,}854$$

É a *média harmónica*: castiga quando uma das duas é muito baixa. **Exemplo:** precision 1,0 e recall 0,1 dão média simples 0,55 (parece razoável), mas F1 = 0,18 (revela o problema). **Analogia:** um aluno com 20 a Matemática e 2 a Português não é "um aluno de 11" — o F1 mostra essa fraqueza.

ROC-AUC (extra)

Mede se o modelo *ordena* bem os clientes por risco, independentemente do limiar. Vai de 0,5 (ao calhas, como atirar uma moeda) a 1,0 (perfeito). **Leitura:** escolhendo ao acaso um cliente que saiu e outro que ficou, há 97,7% de chance de o modelo dar maior probabilidade ao que saiu. O nosso: **0,977**.

Resumo dos nossos números

Métrica	Pergunta curta	Baseline	Regressão Logística
Accuracy	Quantas acertou no total?	0,735	0,925
Precision	Quando dá alarme, acerta?	0,000	0,884
Recall	Apanha quem sai?	0,000	0,825
F1	Equilíbrio dos dois	0,000	0,854
ROC-AUC	Ordena bem por risco?	—	0,977

13. Coeficientes, odds ratio e correlação

O coeficiente é o "peso" que o modelo deu a cada variável. O sinal diz a direção e o tamanho diz a força:

- **Negativo:** quanto maior a variável, *menor* o risco de sair (protege).
- **Positivo:** quanto maior a variável, *maior* o risco de sair.
- **Perto de 0:** quase não influencia.

Como normalizámos as variáveis, cada coeficiente significa "o efeito de subir **1 desvio-padrão**" nessa variável — e por isso podemos compará-los entre si.

Variável	Coeficiente	O que quer dizer
Frequência no mês atual	−4,22	Ir mais vezes agora protege muito
Antiguidade (Lifetime)	−3,76	Clientes antigos são fiéis; os novos são o grupo de risco
Frequência histórica (total)	+3,14	Ler em conjunto com a primeira (ver abaixo)
Idade	−1,18	Mais velhos saem menos
Meses até ao fim do contrato	−0,74	Quem ainda tem muito contrato pela frente fica
Aulas de grupo	−0,73	O lado social prende o cliente
Contrato de 1 mês	+0,66	Contrato mensal = sair é fácil
Género	+0,04	Praticamente irrelevante

O "mistério" do +3,14

Sozinho, o +3,14 parece dizer "quem vai mais ao ginásio sai mais" — o que não faz sentido. A explicação é a **multicolinearidade**: a frequência total e a do mês atual têm correlação de **0,95**, ou seja, contam quase a mesma história.

Quando duas variáveis são quase gémeas, o modelo não as lê separadas: lê a *diferença* entre elas. A leitura correta é: **para a mesma frequência neste mês, quem costumava ir mais tem mais risco.**

Exemplo com duas pessoas:

- **Ana:** sempre foi 2 vezes por semana e este mês foi 2 vezes. Está no seu ritmo normal → risco baixo.
- **Bruno:** sempre foi 4 vezes por semana e este mês foi 2 vezes. Está a perder o hábito → risco alto.

Os dois foram 2 vezes este mês, mas o Bruno caiu para metade. **O sinal de alerta é a queda da frequência face ao hábito do cliente.** É como um amigo que te ligava todos os dias e passou a ligar uma vez por semana: o número absoluto não é baixo, mas a mudança diz muito.

Odds ratio (razão de chances)

Odds (chances) é a forma das apostas: "3 para 1". A odds ratio é o coeficiente transformado com a exponencial (e elevado ao coeficiente) e diz **por quanto se multiplicam as chances** de sair quando a variável sobe 1 desvio-padrão.

- Odds ratio **> 1**: as chances aumentam. Ex.: 1,93 para contrato de 1 mês = chances quase a dobrar.
- Odds ratio **< 1**: as chances diminuem. Ex.: 0,31 para a idade = chances reduzidas a cerca de um terço.
- Odds ratio **= 1**: sem efeito (género: 1,04).

Cuidado: nas variáveis muito correlacionadas (as duas frequências), as odds ratio individuais são exageradas e não devem ser lidas sozinhas. Foi por isso que as interpretámos em conjunto.

Correlação não é causalidade

O modelo mostra *associações*, não causas. "Quem faz aulas de grupo sai menos" não prova que obrigar alguém a fazer aulas de grupo o vai reter — talvez as pessoas mais motivadas é que escolham aulas de grupo. Exemplo clássico: as vendas de gelados e os afogamentos sobem juntos no verão, mas os gelados não causam afogamentos; o calor causa os dois.

14. Reprodutibilidade: random_state, joblib, README e requirements

Reprodutível quer dizer que qualquer pessoa, noutro computador, corre o código e obtém *exatamente* os mesmos resultados. É como uma receita bem escrita: com os mesmos ingredientes e passos, o bolo sai igual em qualquer cozinha. Foi a palavra-chave do enunciado.

Peça	O que é	Analogia	No nosso projeto
`random_state=42`	Fixa os sorteios aleatórios	Baralhar as cartas sempre da mesma forma	Mesma divisão e mesmos folds em todas as execuções
CSV original intocado	Os dados nunca são alterados	Guardar o negativo da fotografia	`data/gym_churn_us.csv` só de leitura
`joblib`	Grava o pipeline treinado num ficheiro	Congelar uma refeição pronta para aquecer depois	`models/pipeline_churn.joblib`

Peça	O que é	Analogia	No nosso projeto
`requirements.txt`	Lista das bibliotecas e versões	A lista de ingredientes com marca e quantidade	pandas 2.3.3, scikit-learn 1.6.1, numpy 2.0.2…
`README.md`	Instruções de como correr o projeto	O manual de instruções do móvel do IKEA	Como instalar, executar e o que é gerado
Corre sem intervenção	Um comando faz tudo do início ao fim	Botão “programa completo” da máquina de lavar	`python pipeline_churn.py` ou “Run all” no Colab

Porque guardar o pipeline inteiro e não só o modelo? Porque um cliente novo chega com dados “crus” (idade 27, não normalizada). Se guardássemos só a Regressão Logística, quem a usasse teria de repetir à mão a imputação, o one-hot e a normalização, com as médias certas do treino — fácil de errar. Com o pipeline inteiro basta `joblib.load(...)` e `predict_proba(...)`.

Como provámos que é reprodutível:

1. O script recarrega o `.joblib` e confirma que dá *exatamente* as mesmas previsões.
2. Criámos um ambiente Python limpo só com o `requirements.txt`, corremos o script e obtivemos os mesmos números (F1 = 0,854).
3. Corremos o notebook numa pasta vazia: ele descarregou o CSV do Kaggle sozinho e deu o mesmo resultado.

Ferramentas usadas: Python (linguagem); **pandas** (ler e manipular tabelas, como um Excel programável); **scikit-learn** (a biblioteca de Machine Learning: Pipeline, ColumnTransformer, LogisticRegression, métricas); **matplotlib** (gráficos); **Colab** (Jupyter Notebook gratuito da Google, no browser).

15. Perguntas prováveis do professor (e respostas)

1. Porque não usaram a accuracy como métrica principal? Porque as classes são desequilibradas (26,5% de churn). O baseline, que diz sempre “fica”, tem 73,5% de accuracy e não deteta ninguém. Recall e F1 da classe churn mostram se o modelo serve para o negócio.

2. Porque deram prioridade ao recall e não à precision? Porque um falso negativo (cliente que sai sem ser detetado) custa a mensalidade perdida; um falso positivo custa só um contacto. Preferimos ligar a mais do que perder clientes.

3. Onde é que o vosso pipeline evita a fuga de informação? Em quatro pontos: limpeza sem estatísticas; divisão antes de transformar; pré-processamento dentro do Pipeline com fit só no treino; e teste usado uma única vez.

4. Para que serve o stratify? Para o treino e o teste terem a mesma proporção de churn (26,5%). Sem isso, uma divisão azarada podia distorcer as métricas.

5. Porque usaram imputação se não havia valores em falta? Para o modelo guardado continuar a funcionar se um cliente novo chegar com um campo vazio. É robustez para o uso real.

6. Porque fizeram one-hot à duração do contrato, se já é um número? Porque 1, 6 e 12 são tipos de plano, não quantidades. Como número, o modelo assumiria que o efeito cresce em linha reta de 1 para 12 meses.

7. Porque normalizaram? Para as variáveis ficarem na mesma escala (idade até 41, gastos até 552) e os coeficientes poderem ser comparados entre si.

8. O coeficiente da frequência histórica é positivo. Isso não é estranho? É efeito da multicolinearidade: está 95% correlacionada com a frequência do mês. Lidas em conjunto, dizem que a queda de frequência face ao hábito é o sinal de alerta.

9. As variáveis do mês atual não são fuga de informação? Só seriam se não fossem conhecidas no momento da previsão. Assumimos que o modelo corre no início do mês para prever o mês seguinte, quando esses dados já existem. É uma limitação que declarámos.

10. Para que serve a validação cruzada se já têm um conjunto de teste? Para confirmar que o resultado não depende da sorte de uma divisão. Os 5 folds deram F1 = 0,858 ± 0,017, muito estável, e o teste confirmou (0,854).

11. Como melhorariam o modelo? Ajustar o limiar ao custo real de cada erro, testar outros algoritmos (ex.: árvores, random forest) e monitorizar o recall todos os meses, retreinando se descer de 0,75.

12. O que faz o random_state=42? Fixa a aleatoriedade: a divisão e os folds são sempre os mesmos, por isso qualquer pessoa obtém os mesmos resultados.

13. Como usaram a IA generativa e como a verificaram? Como apoio ao planeamento, ao código e ao texto. Executámos todo o código, confirmámos os resultados num ambiente limpo e validámos a interpretação dos coeficientes com a análise de correlações.

14. Na prática, como é que o ginásio usaria isto? No início de cada mês carrega o `.joblib`, passa a lista de clientes ativos e recebe a probabilidade de churn de cada um. A equipa contacta primeiro os de maior risco.

16. Glossário rápido

Termo	Em uma frase
Accuracy	Percentagem de previsões certas no total (engana com classes desequilibradas)
Baseline	Modelo ingénuo usado como fasquia mínima de comparação
Churn	Cliente que cancela; taxa de churn = % que cancela
Classificação binária	Prever uma de duas categorias (sai / fica)
Coeficiente	Peso que o modelo dá a uma variável; sinal = direção, tamanho = força
ColumnTransformer	Distribui cada grupo de colunas para o tratamento certo
Correlação (r)	Número de −1 a +1 que mede se duas coisas variam juntas
CRISP-DM	Roteiro de 6 fases para projetos de dados
CSV	Ficheiro de tabela em texto, valores separados por vírgulas
Dataset	Conjunto de dados usado no projeto
Desvio-padrão	Quanto os valores se espalham à volta da média
DummyClassifier	O baseline do scikit-learn; aqui diz sempre “fica”
F1-score	Média harmónica de precision e recall
Falso negativo (FN)	Saiu, mas o modelo disse que ficava (o erro mais caro aqui)
Falso positivo (FP)	Ficou, mas o modelo disse que saía (falso alarme)
Feature / variável	Característica usada para prever (idade, contrato…)

Termo	Em uma frase
fit / transform	fit = aprender com os dados; transform = aplicar o que aprendeu
Fuga de informação (data leakage)	O modelo vê informação que não teria na vida real e parece melhor do que é
Imputação	Preencher valores em falta (mediana, moda)
joblib	Biblioteca que grava e carrega o modelo treinado num ficheiro
Limiar (threshold)	Probabilidade a partir da qual se diz “vai sair” (0,5)
Matriz de confusão	Tabela 2×2 que cruza o real com o previsto
Mediana / moda	Valor do meio / valor mais frequente
Multicolinearidade	Duas variáveis quase iguais que confundem os coeficientes
Normalização (StandardScaler)	Pôr variáveis na mesma escala: (valor − média) / desvio-padrão
Odds ratio	Por quanto se multiplicam as chances quando a variável sobe
One-hot encoding	Transformar uma categoria em várias colunas 0/1
Pipeline	Sequência fixa de passos, do dado bruto à previsão
Precision	Dos alarmes dados, quantos estavam certos
random_state	Semente que torna os sorteios repetíveis
Recall	Dos que saíram, quantos o modelo apanhou
Regressão Logística	Algoritmo de classificação que dá uma probabilidade via curva em S
Reprodutibilidade	Qualquer pessoa obtém os mesmos resultados ao correr o código
ROC-AUC	Quão bem o modelo ordena os clientes por risco (0,5 a 1)
scikit-learn	Biblioteca Python de Machine Learning
Sigmoide	Função em S que converte uma pontuação numa probabilidade

Termo	Em uma frase
Stratify	Manter a mesma proporção de classes no treino e no teste
Treino / teste	Dados para aprender (80%) / dados para avaliar no fim (20%)
Validação cruzada (k-fold)	Repetir treino e teste em k partes para medir estabilidade
Variável-alvo (target)	O que se quer prever: aqui, Churn